Sprint 2 — UDP02

Servidor de Mensagens Fora de Ordem

Explicação do código `UDPServer2.java`

Documento de apoio à defesa

22 de outubro de 2026

Conteúdo

1 Objetivo do servidor **2**

2 Estruturas de dados **2**

3 Fluxo do `main` **2**

4 Método `processDeliveredMessages` **3**

5 Exemplo de cascata **4**

6 Tratamento de duplicados **4**

7 Limitações da solução **5**

8 Terminação do programa **5**

9 Glossário rápido **5**

10 Perguntas prováveis na defesa **6**

1 Objetivo do servidor

O servidor recebe datagramas no formato:

`N,payload`

onde `N` é o número de sequência e `payload` é a mensagem.

O servidor só entrega as mensagens à aplicação quando estas chegam pela ordem correta $(1, 2, 3, \ldots)$. Se uma mensagem chegar fora de ordem, é guardada temporariamente. Quando a mensagem em falta chega, o servidor entrega essa mensagem e todas as que estavam à espera, em **cascata**.

2 Estruturas de dados

`static List<String> finalMessages`
Lista das mensagens **já entregues** pela ordem correta. É o resultado final do protocolo. Nota: uma mensagem pode ter sido recebida fora de ordem e só mais tarde ser entregue; a lista reflete a ordem de *entrega*, não a ordem de chegada à rede.

`static Map<Integer, String> outOfOrderMessages`
Mapa que guarda mensagens **recebidas mas ainda não entregues**.

- Chave: número de sequência (`Integer`).
- Valor: payload (`String`).
- Serve como "sala de espera" para mensagens adiantadas.

`static List<String> deliveredThisStep`
Lista auxiliar que regista as mensagens entregues **durante o processamento de um único datagrama**. Pode conter a mensagem recebida e as que estavam em espera e foram libertadas em cascata. É limpa no início de cada passo. Serve apenas para logging/depuração; não influencia a decisão nem o estado L.

3 Fluxo do `main`

1. Cria um `DatagramSocket` no porto 6789.
2. Inicializa $L = 0$ (última mensagem entregue em ordem). A próxima mensagem esperada é sempre $L + 1$.
3. Entra num ciclo `while(true)` que corre indefinidamente.

Dentro do ciclo:

1. Cria um buffer de 1000 bytes e um `DatagramPacket request`.
2. Chama `socket.receive(request)` — bloqueia até chegar um datagrama.
3. Constrói a string `received` com:

```
new String(request.getData(), 0, request.getLength()).
    trim()
```

- `getLength()` limita a leitura ao número exato de bytes recebidos (evita lixo do buffer).
- `trim()` remove espaços em branco nas extremidades.

4. Limpa `deliveredThisStep` (para não haver resíduos do passo anterior).
5. Procura a primeira vírgula:

```
            int comma = received.indexOf(',');
```

   - Se `comma > 0`, separa: `nStr = received.substring(0, comma).trim()` e `payload = received.substring(comma + 1)`.
   - Se `comma <= 0`, a mensagem é malformada.

6. Tenta converter `nStr` para inteiro N.
7. Chama `L = processDeliveredMessages(L, N, payload)`.
8. Compara o L antigo com o novo:

```
            if (L != oldL)                // houve entrega
            replyStr = received;     // echo do datagrama
            else
            replyStr = "waitingfor," + (L + 1);
```

9. Cria um `DatagramPacket` com `replyStr` para o endereço e porto de origem (`request.getAddress()`, `request.getPort()`).
10. Envia a resposta com `socket.send(reply)`.
11. Imprime informação de debug na consola: `Received`, `Replied`, L, `outOfOrderMessages`, `deliveredThisStep`, `finalMessages`.

Tratamento de erros. Se ocorrer `NumberFormatException` ou faltar a vírgula, responde-se `waitingfor,L+1`. O `catch (IOException)` interno apanha erros de `receive`/`send` e continua o ciclo (não faz `break`).

4 Método `processDeliveredMessages`

Assinatura

```
        public static int processDeliveredMessages(
        int nLastMessageInOrder,    // L atual
        int nCurrentMessage,        // N da mensagem recebida
        String currentMessage)      // payload
```

Contrato

- Recebe o L atual, o N e o payload.
- Devolve o novo L (última mensagem entregue em ordem).
- Pode devolver um valor **maior** que `nCurrentMessage` se houver entrega em cascata.
- Se não houver entrega, devolve o mesmo L que recebeu.

Passos

1. Limpa `deliveredThisStep` (defensivamente).
2. Guarda $L =$ `nLastMessageInOrder`.
3. Se $N = L + 1$:
   - Adiciona `currentMessage` a `finalMessages` e a `deliveredThisStep`.
   - Atualiza $L = N$.
   - Enquanto `outOfOrderMessages` contiver $L + 1$:

```
nextPayload = outOfOrderMessages.remove(L + 1);
finalMessages.add(nextPayload);
deliveredThisStep.add(nextPayload);
L++;
```

4. Senão, se $N > L + 1$:
   - Se `outOfOrderMessages` **não** contiver N: `outOfOrderMessages.put(N, currentMessage)`.
   - Se já contiver, ignora (duplicado na sala de espera).
5. Senão ($N \leq L$): ignora (duplicado de mensagem já entregue).
6. Retorna L.

5 Exemplo de cascata

Sequência de chegada: $1, 3, 4, 5, 2$.

Chega	L **antes**	L **depois**	**Ação**
1	0	1	Entrega 1.
3	1	1	Guarda 3 em `outOfOrderMessages`.
4	1	1	Guarda 4 em `outOfOrderMessages`.
5	1	1	Guarda 5 em `outOfOrderMessages`.
2	1	5	Entrega 2 e, em cascata, 3, 4, 5.

Tabela 1: Exemplo de entrega em cascata. No fim, `finalMessages` fica com $1, 2, 3, 4, 5$.

6 Tratamento de duplicados

Caso A — duplicado de mensagem já entregue
$N \leq L$. Não entra em nenhum ramo de entrega nem de guardar. É ignorado silenciosamente.

Caso B — duplicado de mensagem que está à espera
$N > L+1$. A verificação `if (!outOfOrderMessages.containsKey(N))` impede a reinserção. A cópia duplicada é ignorada; mantém-se a primeira.

Caso C — mensagem repetida com o mesmo N mas payload diferente
O código ignora o novo payload (mantém o primeiro). Não há deteção de inconsistência. É uma limitação.

7 Limitações da solução

- **Estado estático partilhado.** `finalMessages`, `outOfOrderMessages` e `deliveredThisStep` são `static`. Todos os clientes partilham o mesmo estado. Se dois clientes enviarem numerações independentes, misturam-se.
- **Crescimento não limitado.** Se um cliente enviar $N = 1\,000\,000$, a mensagem fica guardada para sempre se as anteriores nunca chegarem. A memória pode esgotar-se.
- **Sem timeout.** Se a mensagem em falta nunca chegar, as mensagens guardadas ficam bloqueadas indefinidamente.
- **Sem separação por cliente.** Seria necessário um `Map<String, EstadoCliente>` com chave `endereço:porto` para isolar cada cliente.
- **Sem deteção de corrupção.** O UDP deteta corrupção via checksum, mas a aplicação não tem mecanismo de recuperação.

8 Terminação do programa

O ciclo `while(true)` **não tem** nenhuma condição de saída no código. Não existe `break`, `return` nem flag de paragem. O programa só termina quando o processo Java for terminado por um sinal externo.

Sinal	Como se envia	Apanhável?	Observações
`SIGINT`	`Ctrl+C`	Sim	Interrupção do teclado. Corre *shutdown hooks*.
`SIGTERM`	`kill <pid>`	Sim	Pedido educado de terminação (padrão).
`SIGHUP`	Fechar terminal / `kill -HUP`	Sim	Também corre *shutdown hooks*.
`SIGKILL`	`kill -9 <pid>`	Não	O processo é morto imediatamente pelo kernel. Sem *cleanup*.

Tabela 2: Sinais que podem terminar o processo.

Detalhe importante. Mesmo que o socket fosse fechado, o `while(true)` continuaria: o `catch (IOException)` interno apanharia a `SocketException` e o ciclo prosseguiria. Portanto, **fechar o socket não termina o programa**. Só um sinal que mate a JVM o faz.

9 Glossário rápido

L Última mensagem entregue em ordem.

$L + 1$ Próxima mensagem esperada.

N Número de sequência da mensagem recebida.

payload Conteúdo da mensagem (depois da vírgula).

echo Resposta que devolve o próprio datagrama recebido (indica que houve entrega).

waitingfor,L+1 Resposta que pede a mensagem em falta.

cascata Entrega consecutiva de várias mensagens guardadas quando a lacuna é preenchida.

10 Perguntas prováveis na defesa

1. **O que acontece se receberes a mensagem 2 duas vezes?**
   Se for a primeira vez, é guardada. Se for a segunda, é ignorada porque `containsKey` já é verdadeiro.

2. **E se receberes a 2 depois de já teres entregue a 2?**
   Como $2 \leq L$, é ignorada (duplicado já entregue).

3. **Explica o contrato de `processDeliveredMessages`.**
   Recebe L, N e payload; devolve o novo L. Pode devolver um valor maior que N se houver cascata.

4. **Porque é que o método devolve o novo L e não N?**
   Porque numa cascata podem ser entregues várias mensagens. O valor devolvido é o último número efetivamente entregue.

5. **Como é que o `main` decide entre `echo` e `waitingfor`?**
   Compara L antes e depois da chamada. Se mudou, houve entrega → `echo`. Se não mudou → `waitingfor,L+1`.

6. **O que acontece se a mensagem esperada nunca chegar?**
   As mensagens fora de ordem ficam presas em `outOfOrderMessages`. Não há timeout. É uma limitação.

7. **Como resolverias o problema de múltiplos clientes?**
   Mantendo um estado por cliente, por exemplo `Map<String, EstadoCliente>` com chave `endereço:porto`.

8. **Porque é que o `remove` em `outOfOrderMessages.remove(L+1)` é importante?**
   Porque devolve o valor e remove a entrada. Sem remover, a próxima iteração voltaria a encontrar a mesma mensagem e entraria em ciclo infinito.

9. **Que sinais podem terminar o servidor?**
   `SIGINT`, `SIGTERM`, `SIGHUP` (apanháveis pela JVM) e `SIGKILL` (não apanhável). O código não tem saída própria.

Fim